Breaking digital signatures from tropical matrix semirings

Alessandro Sferlazza

Technical University of Munich, Germany

Abstract. In the recent preprint [GMS26], Grigoriev, Monico and Shpilrain proposed a digital signature protocol based on the use of matrices over the tropical integer semiring. We show some design flaws of the proposed scheme, together with an efficient attack to forge signatures for an arbitrary message, and a key-recovery attack when given access to a list of honest signatures.

Keywords: digital signature · tropical semiring · cryptanalysis

1 Introduction

A recent work [GMS26] by Grigoriev, Monico and Shpilrain proposes to build a cryptographically secure digital signature scheme from matrices over a tropical semiring.

In this note, we show a simple forgery and several key-recovery attacks against the scheme, providing Python code to show the feasibility of the attacks. Furthermore, we point out some problems in the design of the protocol. First, the proposed signature scheme has a nonzero probability to reject honest signatures. Second, the security of the signature scheme does not rely on the claimed underlying hard problem.

On the same note, we observe that the underlying problem is claimed to to be hard to solve since it is provably NP-hard. However, the NP-hardness is usually unrelated to the security of a cryptographic scheme: indeed, a scheme is secure if the *concrete* instances of the underlying hard problem (relative to concrete sizes and parameters) are *almost always* difficult to solve, while NP-hardness is an asymptotic and worst-case notion. With our implementation, we show indeed that even a brute-force key-recovery attack from a given public key is feasible with the proposed parameters.

Acknowledgements. I wish to thank Lorenz Panny for the support and helpful discussions that lead to this short note.

2 Preliminaries

In this section, we quickly review the constructions in [GMS26].

2.1 Tropical matrices

Suppose given a linearly ordered group $(R, \leq, +)$. The associated *tropical semiring* $R_{\min,+} = (R \cup \{\infty\}, \oplus, \otimes)$ is taken to be the extended line $R \cup \{\infty\}$ equipped with the binary operations $x \oplus y = \min\{x, y\}$ and $x \otimes y = x + y$, with neutral elements respectively ∞ and 0. The operations $\oplus$ and $\otimes$ satisfy commutativity, associativity and distributivity.

E-mail: alessandro.sferlazza@tum.de (Alessandro Sferlazza)

Date of this document: 2026-02-19.

The semiring operations can be naturally extended to *matrices* with coefficients in the tropical semiring $S = R_{\min,+}$, letting $\oplus$ act component-wise and defining $\otimes$ via the usual row-column product. More precisely, given $A, B \in S^{m\times n}$, we define the sum $M = A \oplus B$ to be the matrix with entries $M_{i,j} = A_{i,j} \oplus B_{i,j} = \min\{A_{i,j}, B_{i,j}\}$. Given $A \in S^{m\times k}$ and $B \in S^{k\times n}$, we define the product as $M = A \otimes B$ with entries $M_{i,j} = \bigoplus_{\ell=1}^{k}(A_{i,\ell} \otimes B_{\ell,j}) = \min_{\ell\in\{1,\dots,k\}}(A_{i,\ell} + B_{\ell,j})$.

2.2 The signature scheme

We now give an overview of the signature protocol proposed in [GMS26]. Consider the tropical semiring $S = \mathbb{Z}_{\min,+}$ associated with the integer ring. For $r, s \in \mathbb{Z}$, let $\mathcal{M}_{r,s}$ denote the set of matrices over S with r rows and s columns whose entries lie in the range $\mathbb{Z} \cap [0, 255]$. More generally, let $\mathcal{M}_{r,s}^{\le N}$ denote the set of r-by-s integer matrices with entries in $[0, N]$. Fix parameters $(m, n, k) = (8, 8, 7)$.

The signer owns a private-public key pair $(\mathsf{sk}, \mathsf{pk})$ where we set $\mathsf{sk} = (X, Y)$ with $X \xleftarrow{\$} \mathcal{M}_{m,k}$ and $Y \xleftarrow{\$} \mathcal{M}_{k,n}$ sampled uniformly at random, and $\mathsf{pk} = T := X \otimes Y \in \mathcal{M}_{m,n}^{\le 510}$.

An honest signature for a (timestamped) message m is produced as follows. Let $\mathcal{H}$ be a cryptographically secure hash function that maps bit strings to $\mathcal{M}_{n,m}$, and define $M = \mathcal{H}(m)$. Sample $U \xleftarrow{\$} \mathcal{M}_{n,k}$ and $V \xleftarrow{\$} \mathcal{M}_{k,m}$ uniformly at random, and keep sampling until we have $(M \otimes X) \oplus U \neq U$ and $(Y \otimes M) \oplus V \neq V$. The signature is the tuple

$$\sigma = \big(M, (M \otimes X) \oplus U, (Y \otimes M) \oplus V, X \otimes V, U \otimes Y, U \otimes V\big).$$

In the verification algorithm, a signature $\sigma = (M, A, B, P, R, S)$ for a message m is considered valid for the public key T if $M = \mathcal{H}(m)$ is the correct hash and if the following conditions hold:

$$A \otimes B \;=\; (M \otimes T \otimes M) \oplus (M \otimes P) \oplus (R \otimes M) \oplus S, \tag{V1}$$

$$A \otimes B \;\neq\; S. \tag{V2}$$

3 Breaking the signature

In this section, we describe how the proposed signature scheme doesn't achieve correctness nor security. We follow [Gal12, §1.3.2] for general constructions and requirements of public-key signature schemes.

3.1 Rejection of honest signatures

Given a signing algorithm $\mathsf{Sign}(m, \mathsf{sk}) \mapsto \sigma$ and a verification algorithm $\mathsf{Verify}(m, \sigma, \mathsf{pk}) \mapsto \{\mathsf{valid}, \mathsf{invalid}\}$, verification is supposed to accept an honestly generated signature, i.e., we require that for all messages m we have

$$\mathsf{Verify}\big(m, \mathsf{Sign}(m, \mathsf{sk}), \mathsf{pk}\big) = \mathsf{valid}.$$

We show that this does not hold in the proposed scheme: an honest signature can be rejected, even though with very low probability.

Suppose given a timestamped message m and a secret key (X, Y). Say the randomly sampled $U \in \mathcal{M}_{n,k}$ has very low entries concentrated on the ℓ_0-th column for some $\ell_0 \in \{1, \dots, k\}$, and similarly $V \in \mathcal{M}_{k,m}$ has very low entries on the ℓ_0-th row.

More precisely, say for all $i \in, j \in \{1, \dots, m\}$ the random sampling satisfies

$$\forall i \in \{1, \dots, n\} \qquad U_{i,\ell_0} = \min_{\ell\in\{1,\dots,k\}} U_{i,\ell} \;\le \min_{j\in\{1,\dots,m\}} M_{i,j}$$

$$\forall j \in \{1, \dots, m\} \qquad V_{\ell_0,j} = \min_{\ell\in\{1,\dots,k\}} V_{\ell,j} \;\le \min_{i\in\{1,\dots,n\}} M_{i,j}$$

and suppose further $A := (M \otimes X) \oplus U \neq U$ and $B := (Y \otimes M) \oplus V \neq V$, that is, at least one entry of U (resp. V) is larger than the corresponding entry in $M \otimes X$ (resp. $Y \otimes M$), so that the matrices U, V are not rejected during the signing algorithm.

Even though the signature is regularly carried out with U, V, the verification rejects the signature because of the equality $A \otimes B = U \otimes V$. Indeed, for $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, m\}$, the above conditions imply

$$U_{i,\ell_0} = \min_{\ell} A_{i,\ell} = \min_{\ell} U_{i,\ell}, \quad V_{\ell_0,j} = \min_{\ell} B_{\ell,j} = \min_{\ell} V_{\ell,j}$$

and these equalities in turn imply

$$(A \otimes B)_{i,j} = \min_{\ell \in \{1,\dots,k\}} (A_{i,\ell} + B_{\ell,j}) = U_{i,\ell_0} + V_{\ell_0,j} = \min_{\ell \in \{1,\dots,k\}} (U_{i,\ell} + V_{\ell,j}) = (U \otimes V)_{i,j}.$$

An easy countermeasure for the issue we just presented is to fix the rejection sampling criteria in the signing algorithm. Namely, while sampling U and V in Sign, replace the conditions $A \neq U$ and $B \neq V$ with the stronger condition $(A \otimes B) \neq (U \otimes V)$, equivalent to condition (V2) in Verify. With this modification, an honest signature cannot be rejected by the verification algorithm: (V2) tautologically holds since it is checked at signing time, and the equality condition (V1) is always verified due to distributivity of $\oplus, \otimes$.

3.2 Target-message forgery

The rejection sampling criteria mentioned in Section 3.1 and the requirement $A \otimes B \neq S$ (V2) were chosen in [GMS26] to prevent the following trivial attack: in the signature (M, A, B, P, R, S), the adversary would choose $(A, B) = (U, V)$ with very low entries (say, for example, all zero) and $S = U \otimes V$ so that the equality condition (V1) becomes $A \otimes B = S$ and is trivially satisfied.

We now show how to slightly tweak this attack in order to bypass the countermeasure above. The following procedure allows an attacker to sign an arbitrary (timestamped) message in constant time.

We exploit the fact the requirement $A \otimes B \neq S$ is quite weak: if the two matrices differ in at least one component, the check passes. Therefore, we can simply choose S to be the zero matrix *except for one entry*, and recover $A \otimes B$ to pass verification accordingly.

More precisely, let m be an arbitrary timestamped message, and hash it into a matrix $M = \mathcal{H}(m)$. Define the matrices $S \in \mathcal{M}_{n,m}^{\leq 510}$, $P \in \mathcal{M}_{m,m}^{\leq 510}$ and $R \in \mathcal{M}_{n,n}^{\leq 510}$ with entries

$$S_{i,j} = \begin{cases} 510 & \text{if } i = j = 1 \\ 0 & \text{otherwise,} \end{cases} \qquad P_{i,j} = 0 \ \ \forall i, j, \qquad R_{i,j} = 0 \ \ \forall i, j.$$

Let W denote the matrix $(M \otimes T \otimes M) \oplus (M \otimes P) \oplus (R \otimes M) \oplus S$. By construction, we have $W_{i,j} = 0$ for $(i, j) \neq (1, 1)$, and $W_{1,1} = \min\{M_{i,j} \mid i = 1 \text{ or } j = 1\}$.

We build $A \in \mathcal{M}_{n,k}, B \in \mathcal{M}_{k,m}$ as follows to satisfy $W = A \otimes B$, i.e., verification condition (V1):

$$A_{i,j} = \begin{cases} W_{1,1} & \text{if } i = j = 1 \\ 0 & \text{otherwise,} \end{cases} \qquad B_{i,j} = \begin{cases} W_{1,1} & \text{if } i > 1 \text{ and } j = 1 \\ 0 & \text{otherwise.} \end{cases}$$

By construction, $(A \otimes B)_{1,1} = W_{1,1}$ lies in $[0, 255]$ while $S_{1,1} > 255$, so $A \otimes B \neq S$ (i.e., verification condition (V2)) always holds. In other words, the verification algorithm accepts the forgery $\sigma = (M, A, B, P, R, S)$ for the message m with probability 1.

4 Key-recovery attacks

In this section, we present some feasible key-recovery attacks that break the protocols in [GMS26].

Recall from above that we take as secret key a pair $(\mathsf{sk}, \mathsf{pk})$ where the secret key $\mathsf{sk} = (X, Y)$ is a pair of integer matrices $X \in \mathcal{M}_{m,k}$ and $Y \in \mathcal{M}_{k,n}$ with entries in $[0, 255]$ and the public key $\mathsf{pk} = T = X \otimes Y$ is the tropical product of the secret matrices.

Performing a key-recovery attack on a given public key T amounts to factoring T into *any* pair of matrices (X, Y) with dimensions as above, since any (X, Y) with $X \otimes Y = T$ would make it possible for an attacker to produce a valid signature. We remark that a solution to this problem need not be unique. On the contrary, our experiments (see Table 1 below) show that matrices $\widetilde{X}, \widetilde{Y}$ that differ from given X, Y on a significant portion of the entries can still satisfy $\widetilde{X} \otimes \widetilde{Y} = X \otimes Y$.

The authors of [GMS26] claim that security of the proposed schemes relies on the factorization problem of a tropical matrix. We remark however that only *key recovery* relies on factoring T, whilst signature unforgeability is a priori easier. Indeed, a signature (M, A, B, P, R, S) exposes matrices A, B that are not built built as a product, but rather as a *sum* of tropical matrices. We exploit this in Section 4.2 for efficient key recovery.

4.1 Brute-force attacks

A trivial attack to try is brute-forcing the keys from the given public data. We implemented two versions of this attack in Python (see Section 5 below), using the interface to the z3 SMT solver [DB08].

- The first version takes as input a public key $T \in \mathcal{M}_{m,n}^{\leq 510}$ and finds $X \in \mathcal{M}_{m,k}, Y \in \mathcal{M}_{k,n}$ whose tropical product is T.
- The second version takes T and an honest signature (M, A, B, P, R, S) for a message m and finds valid private keys *and* session keys, i.e., finds (X, Y, U, V) satisfying

$$\begin{cases} X \otimes Y = T, & X \otimes V = P, & Y \otimes U = R, & U \otimes V = S, \\ (M \otimes X) \oplus U = A, & & (Y \otimes M) \oplus V = B. \end{cases}$$

For the given parameters $(m, n, k) = (8, 8, 7)$, both versions of the attack take less than a minute. However, experiments suggest that this brute-force attack has an exponential-time behaviour in (m, n, k). Using larger values of the parameters (m, n, k) is an easy countermeasure against against this attack, and has a minimal impact on performance due to the efficiency of tropical operations.

4.2 Offline attack from collection of signatures

In [GMS26, § 6.2], the authors wonder whether it is possible to recover the secret key given valid signatures produced from the same key pair on many different messages. We now give a positive answer to the above: we present such an attack, and demonstrate its feasibility and scalability via concrete experimental data.

Suppose given N_{sig} honest random signatures $\sigma_r = (M^{(r)}, A^{(r)}, B^{(r)}, \dots)_{r \in \{1,\dots,N_{\text{sig}}\}}$ on pairwise distinct timestamped messages $m^{(r)}$ and let $(U^{(r)}, V^{(r)})_r$ be the corresponding nonce matrices, so that for all r we have $A^{(r)} = (M^{(r)} \otimes X) \oplus U^{(r)}$ and $B^{(r)} = (Y \otimes M^{(r)}) \oplus V^{(r)}$.

The attack uses the following idea: despite the randomness of $U^{(r)}, V^{(r)}$, the matrix $A^{(r)}$ leaks a constant fraction of the entries of $M^{(r)} \otimes X$; similarly, $B^{(r)}$ leaks entries of $Y \otimes M^{(r)}$.

Moreover, $M^{(r)}$ is always public. Since tropical matrix division $(M \otimes X, M) \mapsto X$ is an easy problem, given many signatures we can reasonably hope to recover valid X and Y.

More precisely, for every signature σ_r we have $A^{(r)}_{i,j} \leq (M^{(r)} \otimes X)_{i,j} = \min_\ell(M_{i,\ell} + X_{\ell,j})$ for all $i \in \{1, \dots, n\}$, $j \in \{1, \dots, k\}$. Reversing the inequality we get

$$X_{\ell,j} \geq \max_{r,i} \left(A^{(r)}_{i,j} - M^{(r)}_{i,\ell}\right).$$

with equality iff for at least one (r, i) we have $U^{(r)}_{i,j} - M^{(r)}_{i,\ell} \geq X_{\ell,j}$ and $M^{(r)}_{i,\ell'} - M^{(r)}_{i,\ell} \geq X_{\ell,j} - X_{\ell',j}$ for all $\ell' \neq \ell$. Note that the above inequalities are verified with very high probability for low values of $X_{\ell,j}$, and the low-value entries of X, Y are the ones determining the entries of the product $T = X \otimes Y$.

Therefore, we set $\widetilde{X}_{\ell,j} := \max_{r,i}(A^{(r)}_{i,j} - M^{(r)}_{i,\ell})$; similarly set $\widetilde{Y}_{i,\ell} := \max_{r,j}(B^{(r)}_{i,j} - M^{(r)}_{\ell,j})$. Table 1 below shows that the product $\widetilde{T} = \widetilde{X} \otimes \widetilde{Y}$ equals the given public key T in almost all entries. The indices (i, j) where $\widetilde{T}_{i,j} \neq T_{i,j}$ don't coincide imply some entries in the i-th row of $\widetilde{X}$ and the j-th column of $\widetilde{Y}$ are not correct: if $\widetilde{X}_{i,\ell} + \widetilde{Y}_{\ell,j} < T_{i,j}$ the components $(\widetilde{X}_{i,\ell}, \widetilde{Y}_{\ell,j})$ cannot match $(X_{i,\ell}, Y_{\ell,j})$ as that would contradict the identity $X \otimes Y = T$. As a solution, we set the entries $(\widetilde{X}_{i,\ell}, \widetilde{Y}_{\ell,j})$ as unknowns whenever $\widetilde{X}_{i,\ell} + \widetilde{Y}_{\ell,j} < T + \delta$ (allowing for a small $\delta > 0$ to account for a larger margin of error), and recover their value via an SMT solver, to finally satisfy $\widetilde{X} \otimes \widetilde{Y} = T$ for all entries.

Feasibility of the attack follows from the fact that we use $O(\max\{n, m\} \cdot N_{\text{sig}})$ bit operations to recover $\widetilde{X}, \widetilde{Y}$; the subsequent SMT solution step is quite fast, as the number of unknowns is very low compared to the total number of entries in (X, Y).

Table 1: Average data over 20 experiments (in wall-clock seconds) for the attack in Section 4.2 for various values of m and N_{sig}, with $n = m$ and $k = m - 1$. Using z3 version 4.15.6 on an Intel Core Ultra 7 165U, clock speed 1.7MHz, turbo boost disabled.

(m, N_{sig})	(8, 128)	(8, 1000)	(25, 128)	(25, 1000)	(25, 5000)
time (wall-clock seconds)	0.07	0.36	98	33.5	82
% correct entries of $\widetilde{X}, \widetilde{Y}$	78	86	56	66	68
% correct entries of $\widetilde{T}$	95	99.8	97	99.3	99.8

5 Implementation and performance

We provide a Python implementation for all the attacks discussed in Sections 3.2 and 4 at the following address.

https://github.com/sferl/tropmat-attacks

We use the interface and algorithms provided in the proof-of-concept implementation in [GS25] for key generation, signing and verification, using the parameters suggested in [GMS26]. As mentioned above, the attacks in Section 4 make use of the Python interface to the z3 SMT solver [DB08] to partially recover the factorization of a given tropical matrix.

Moreover, we provide code for an example of rejection of an honest signature, to concretely show the situation discussed in Section 3.1.

Forging a signature of a given message as in Section 3.2 takes approximately 0.1 milliseconds. The brute-force attacks of Section 4.1 with the parameters suggested in [GMS26] both take between 40 and 60 seconds on average. The attack of Section 4.2 given the challenge data in [GS25, `chall.py`] recovers the secret key in approximately 0.3 seconds.

References

[DB08] Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008. URL: https://github.com/Z3Prover/z3.

[Gal12] Steven D. Galbraith. *Mathematics of Public-Key Cryptography*. Cambridge University Press, 2012. URL: https://math.auckland.ac.nz/~sgal018/crypto-book/crypto-book.html.

[GMS26] Dima Grigoriev, Chris Monico, and Vladimir Shpilrain. Tropical cryptography IV: digital signatures and secret sharing with arbitrary access structure. Cryptology ePrint Archive, Paper 2026/095, 2026. URL: https://eprint.iacr.org/2026/095.

[GS25] Dima Grigoriev and Vladimir Shpilrain. `gs_trop_sigs`: A basic implementation of the signature and secret-sharing schemes described in [GMS26], 2025. URL: https://github.com/AssociateDeadWood/gs_trop_sigs. Accessed: February 2026.